

Orbital

Live Aircraft, Ship and Satellite Tracking — Project Report

Phone Sett Paing Kyaw (6708369)

Bhone Pyae Hein (6708381)

Han Phyo Htet (6708463)

Nyan Lin Htet (6708397)

CSC480 — 7 September 2026

Table of Contents

Chapter 1: Introduction	2
1.1 Background and Problem Statement	2
1.2 Project Objectives	3
1.3 Project Scope	3
1.4 Expected Benefits.....	5
1.5 Tools and Technologies Used	5
1.6 Scale of the Delivered System	6
Chapter 2: Feasibility Study and Related Work	8
2.1 Technical Feasibility	8
2.2 Economic Feasibility.....	9
2.3 Operational Feasibility.....	10
2.4 Schedule Feasibility	11
2.5 Related Systems and Research	12
2.6 Relevant Theories and Technologies	13
Chapter 3: Requirements Analysis	15
3.1 Stakeholder Analysis	15
3.2 Functional Requirements	16
3.3 Non-Functional Requirements	19
3.4 Use Case Diagram.....	22
3.5 Use Case Descriptions	22
3.6 Software Requirements Specification (SRS)	26

Chapter 1: Introduction

1.1 Background and Problem Statement

At any moment there are roughly fourteen thousand aircraft in the air, tens of thousands of ships at sea, and several thousand satellites in orbit, and almost all of them are broadcasting where they are. Aircraft transmit ADS-B, ships transmit AIS, and satellite orbits are published as orbital elements by government catalogues. The data is public, continuous, and free at the point of collection.

It is not, however, easy to *use*. The problem this project addresses is not that the information is unavailable but that it arrives in a form no ordinary viewer can read, and that the services which package it usefully are commercial.

Four specific difficulties motivate Orbital.

The raw formats are unreadable without domain knowledge. ADS-B reports altitude in feet and speed in knots; AIS encodes “not available” as a number inside the valid range, so a vessel that has not reported its speed transmits 102.3 knots rather than a blank; orbital elements are a two-line text format that has to be propagated through a physics model before it means a position. None of these can be displayed as received.

Every free source is incomplete, and they are incomplete in different places.

Measured directly during this project on identical search areas, OpenSky reported 22 aircraft over Myanmar where adsb.lol reported 4, while adsb.lol reported 33 over inland China where OpenSky reported none. For ships the gap is starker still: the global AIS stream used here returns 9,202 vessels in northern Europe and **two** in the Indian Ocean, because coverage follows volunteer receivers rather than shipping.

Free access is metered, and the meter is easy to exhaust. The OpenSky Network bills by geographic area against a daily credit allowance; adsb.lol permits a burst of four requests and then roughly one every twelve seconds. Naive polling exhausts either within hours. A design that puts the browser in direct contact with these services fails as soon as more than one person opens it.

The commercial alternatives are closed. The vessel-tracking market consolidated during this project’s lifetime — MarineTraffic, FleetMon and Spire Maritime are now owned by Kpler, and ORBCOMM’s AIS business by S&P Global — and the surviving free tiers are credit-metered previews.

The problem statement, stated plainly. Public position data for aircraft, ships and satellites is abundant and free, but it is fragmented across sources with different formats, different coverage gaps and different access limits, and no single free tool presents all three honestly on one map. Orbital is an attempt to build that tool, and to be explicit at every point about what it does *not* know.

That last clause is the part this project treats as a requirement rather than a courtesy. A map that draws a marker where an aircraft was five minutes ago, with no indication that the position is old, is not merely incomplete — it is making a false statement that a viewer has no way to detect.

1.2 Project Objectives

The project's objectives are stated below in the order they were undertaken. Each is measurable, and the evidence for each is recorded in `docs/test-plan.md` and `docs/decisions.md`.

01 — Present live aircraft positions on an interactive Earth in a browser. The viewer can rotate and zoom from a whole-globe view down to street level, select an aircraft, and read its details.

02 — Isolate the browser from every external data source. All upstream communication passes through the project's own backend, so that quota and rate limiting are enforced in exactly one place and a hundred open tabs cost the same as one.

03 — Survive upstream failure without breaking the display. When a data source is unreachable, the system continues to serve the last known state, marked explicitly as stale, rather than returning an error or an empty map.

04 — Normalise every source onto one shared data shape. A single object definition, enforced by types on both sides of the network boundary, so that adding a data source is a backend change that the frontend never learns about.

05 — Extend the same architecture to a second and third class of object. Satellites, whose positions are *computed* rather than observed, and ships, which are observed by a different protocol with different failure modes. This objective exists to test whether the abstraction in 04 was real or accidental.

06 — Establish user accounts and a differentiated service tier. A free tier supported by advertising and a premium tier with a wider capability, so the system has an economic model rather than only a technical one.

07 — Keep the reasoning behind every significant decision on the record. Each choice recorded with its alternatives, its measurements and its cost, so that the design can be defended and so that a decision made on a measurement can be revisited when the measurement changes.

1.3 Project Scope

1.3.1 In scope

Area	What is included
Aircraft	Live positions from <code>adsb.lol</code> and the OpenSky Network, merged; observed

layer	flight track; airline and aircraft type; airport search; receiver-coverage annotation
Satellite layer	~1,400 satellites propagated from published orbital elements; orbit classification; ground track
Ship layer	~23,000 vessels from Fintraffic's Digitraffic feed and aisstream.io, merged; vessel type, dimensions, destination and navigational status
Solar system view	The eight planets and the Moon, to scale in distance, navigable and visitable
Accounts	Registration, sign-in, session management, three tiers (free, premium, admin)
Time travel	Historical satellite positions within an entitlement-limited window

1.3.2 Explicitly out of scope

The following were considered and deliberately excluded. Each exclusion is recorded with its reasoning, and three of them are enforced by automated tests that fail if the boundary is crossed without a written decision.

Excluded	Why
Space debris and rocket bodies	The satellite catalogue holds roughly 100,000 objects, most of which are neither satellites nor meaningful to draw. Enforced by <code>test_no_provider_serves_debris_or_rocket_bodies</code>
Conjunction, collision or re-entry prediction	The propagation model used is accurate to kilometres, and a viewer could not detect a wrong answer. Enforced by <code>test_no_prediction_endpoint_exists</code>
A fourth object class	Any new layer requires an explicit written decision. Enforced by a test on each side of the network boundary, which has now fired twice — once when satellites were added and once when ships were
Ground-station passes and look angles	Feasible, but outside the stated objectives
Flight or voyage prediction	Orbital reports observations; it does not forecast them

1.3.3 A scope boundary that moved, and how

Satellites were removed from the plan early in the project and reinstated later. The mechanism matters more than the outcome: three tests existed whose only purpose was to fail if a satellite ever appeared, so the boundary could not be crossed quietly. When the scope was reinstated, those tests failed, and they were re-aimed at the new boundary rather than deleted — a guard removed the moment it fires was never a guard.

1.4 Expected Benefits

For the general viewer. A single free view of aircraft, ships and satellites, with no account required to look and no advertisement of completeness the data cannot support. The interface states its own limits: the ship layer is subtitled “ships in coastal waters” because both AIS feeds are received from the shore, and a marker whose position is more than two minutes old visibly fades rather than silently persisting.

For the upstream data providers. A client that behaves well. All external traffic is consolidated into one backend making a fixed number of requests regardless of how many people are watching, polling each source at the cadence that source asks for. This is a direct benefit to the volunteer receiver networks the project depends on.

For the project team. A worked example of a system with three interchangeable data layers, in which the interchangeability was tested twice by actually adding a layer. The second and third layers each cost approximately one provider module and one line in a registry, and the polling logic was not modified for either.

For future maintainers. 166 recorded decisions, each with its alternatives and its measurements, and a documented instance in which a decision correct at the time became wrong when the data it was based on changed — with the cost of that made visible.

1.5 Tools and Technologies Used

1.5.1 Core stack

Layer	Technology	Reason
Map rendering	MapLibre GL JS	Renders a globe when zoomed out and a street map when zoomed in, on real satellite imagery, in one continuous view
3D models	three.js	The selected aircraft is drawn as a real airframe inside MapLibre’s own WebGL context via its custom-layer hook
Frontend	React, TypeScript, Vite	The shared data contract is enforced at compile time on the client side
Backend	FastAPI, Python	Pydantic makes the same contract executable and self-documenting on the server side
HTTP client	httpx	Asynchronous, so polling never blocks an API request
WebSocket client	websockets	Required for the global AIS stream, which pushes rather than answers
State	Zustand	Small enough not to need explaining; sufficient for one store
Storage	In-process	One process and one poller — an external cache

	dictionaries; SQLite for accounts	would be operational cost for no benefit. Accounts are the only state that must survive a restart
Orbital mechanics	sgp4	The standard propagator for the element format the catalogues publish
Testing	pytest, Vitest	Both suites run offline against committed fixture data

1.5.2 External data sources

Source	Provides	Licence and access
adsb.lol	Aircraft positions	Open Database Licence; no key, community-operated
OpenSky Network	Aircraft positions	Free for non-commercial use; OAuth2; metered by area
Digitraffic (Fintraffic)	Ship positions, northern Baltic	CC BY 4.0, commercial use permitted ; no key
aisstream.io	Ship positions, global	Free, key required; commercial terms unanswered
CelesTrak / SatNOGS	Orbital elements	Free; SatNOGS is the fallback when CelesTrak is unavailable
JPL Horizons	Lunar spacecraft ephemerides	Free, no key
NASA GIBS, OpenFreeMap	Satellite imagery and vector map tiles	Open

1.5.3 Development environment

Python 3.14 and Node.js, with the backend served by uvicorn and the frontend by Vite. Version control is Git. The system runs entirely offline against a committed fixture provider replaying 157 synthetic aircraft, so that neither development nor the test suite consumes a third-party quota.

1.6 Scale of the Delivered System

For reference in the chapters that follow:

Measure	Value
Backend source	~19,100 lines across 85 Python files
Frontend source	~28,500 lines across 159 TypeScript files
Automated tests	1,842 — 823 backend, 1,019 frontend
HTTP endpoints	16
Recorded decisions	166

Objects served, measured live

13,829 aircraft · 23,297 ships · 1,431 satellites

Chapter 2: Feasibility Study and Related Work

2.1 Technical Feasibility

Technical feasibility was not assessed by argument. Every claim below was tested against the live services before the design depended on it, and two of the findings changed the design.

2.1.1 Can the data be obtained at all?

Question	Method	Result
Are live aircraft positions obtainable without payment?	Called both APIs directly	Yes. OpenSky returned 11,651 aircraft worldwide; adsb.lol returned 10,009
Can the whole world be fetched economically?	Measured a single 6,000 nautical-mile query	Yes — 10,013 aircraft in 1.9 seconds
Can satellite positions be computed rather than fetched?	Propagated published elements offline	Yes. ~1,400 satellites, no network in the request path
Are live ship positions obtainable without payment?	Called Digitraffic and aisstream	Yes. 916 vessels in 37 KB from one request; 17,848 in four minutes from the stream

2.1.2 Two findings that changed the design

One global query is not enough, and a careless measurement said it was. A single 6,000 nm circle returned 10,013 aircraft and four circles returned 10,009, which reads as proof that one suffices. It was not: the other three sample points were inside the area the first already covered. Asking the obvious follow-up question — what is over Australia? — returned **zero** from the single circle and **27** from a direct query. The global sweep is four circles.

This is the most important methodological finding in the project and it recurs in Chapter 7: *a benchmark that samples only where the system already works proves nothing.*

A rate limit cannot be characterised with a burst. adsb.lol appeared to tolerate any request rate when tested with occasional bursts of four. Under continuous load it refused 38% of requests. Measured properly — steady sending, nothing else running — the limit is **four requests, then approximately one every twelve seconds.** Allowing a burst is precisely what a token bucket is for, and mistaking the burst for the limit cost three separate “fixed” claims.

2.1.3 Performance feasibility

The backend runs a single-threaded event loop, so any slow synchronous operation delays every other request and the poller with it. Measured medians:

Objects held	Poll apply	Bounding-box filter	Thin to 2,000	Search
2,000	0.7 ms	0.57 ms	0.82 ms	1.6 ms
10,000	7.4 ms	3.2 ms	9.1 ms	8.3 ms
30,000	14.4 ms	9.2 ms	43.5 ms	24.2 ms

Rendering was measured in the browser at the 2,000-marker cap: **0.80 ms per update, 4.8% of a 60 fps frame budget**, scaling linearly to 10,000 markers — more aircraft than are reported globally.

Technical feasibility: confirmed. Every capability the objectives require was demonstrated on live services before being designed around.

2.2 Economic Feasibility

2.2.1 Direct cost

The direct monetary cost of building and running Orbital is zero. Every component is free at the point of use:

Cost category	Amount	Note
Data sources	0	All six are free; two require a key, neither charges
Map imagery and tiles	0	NASA GIBS and OpenFreeMap are open
Libraries and tooling	0	All open source
Hosting during development	0	Runs locally
Total	0	

The genuine cost is labour: four students over one semester.

2.2.2 The quota is the real budget

Although no money changes hands, one resource is finite and had to be budgeted like money. The OpenSky Network allocates a daily credit allowance, spending credits by the geographic area of each query.

The polling strategy is derived arithmetically from that allowance rather than chosen by feel. In the configuration used, the two scheduled jobs project **3,072 credits per day against a 4,000 allowance — 77%, leaving 23% headroom** for the per-selection lookups a user triggers by clicking. The system refuses to start if the configured intervals would exceed 85% of the allowance, so a mistaken interval fails immediately rather than exhausting the day's credits by mid-afternoon.

Adding the free adsb.lol feed as the primary source, with OpenSky supplementing it every 120 seconds, decoupled the refresh rate from the credit ladder entirely. Ships and satellites

cost nothing at all: Digitraffic and aisstream are unmetered, and satellite positions are computed rather than fetched.

2.2.3 Licensing constrains the revenue model, and unevenly

This is the finding with the clearest commercial consequence, and the three sources sit at three different points:

Source	Commercial use	Consequence
Digitraffic (Fintraffic)	Explicitly permitted (CC BY 4.0)	Usable under any model, with attribution
OpenSky Network	Explicitly forbidden — non-commercial only	Any paid tier must not depend on it
aisstream.io	Unanswered	Asked publicly on their issue tracker in April 2026; no reply. Used on the footing of a free public service, credited, and behind a configuration switch that disables it the day anything is charged for
adsb.lol	ODbL — permits commercial use	A produced work may be licensed freely; a derivative <i>database</i> must be shared alike

The practical conclusion: **a commercial version of Orbital is possible but not with its current source mix**. The aircraft layer would need to drop OpenSky and rely on adsb.lol alone, accepting the coverage gaps that Chapter 2.1 measured. This is recorded as a finding rather than resolved, because resolving it is a business decision and not a technical one.

2.2.4 The implemented model

A free tier supported by advertising and a paid tier are both implemented. The advertisements are **house advertisements only** — the project’s own messages in real advertisement dimensions — because a third-party advertising network would introduce tracking into a project that collects no analytics.

The paid tier is honest about not being paid: the button reads “**Switch premium on**”, **never “Buy”**, and an automated test enforces that wording. There is no payment processor, and a Buy button that takes no money is a lie told in the interface.

2.3 Operational Feasibility

2.3.1 Can it be operated?

Orbital runs as two processes — an API server and a static frontend — with no database server, no message queue and no external cache. Application state lives in process memory; the only state that must survive a restart is accounts, held in a single SQLite file.

Deliberate simplicity. An external cache was considered and rejected: there is one process and one poller, so a shared cache would add an operational component without removing any problem.

2.3.2 Does it survive its dependencies failing?

This was the objective most at risk, because the project depends on six external services it does not control. The failure behaviour is structural: **the API layer never calls an upstream service**. It reads a cache the ingestion layer fills on a schedule. There is therefore no upstream call in the request path that *can* fail.

Verified rather than assumed, including on a day when it was not a drill:

Failure	Behaviour
Upstream returns 5xx	Last good snapshot served, flagged <i>stale</i> ; poller backs off
Upstream rate-limits	Same, using the upstream's own retry hint
One of two aircraft feeds down	The other continues; the map is drawn
One of two ship feeds down	The other continues
CelesTrak returned 503 for a full day	The SatNOGS fallback served; cached elements remain usable for days
Network entirely absent	The fixture provider replays committed data

2.3.3 Can a viewer understand it?

Operational feasibility includes whether the output is *readable*, and this drove a class of requirement that a purely technical analysis would miss.

The interface must not make claims the data cannot support. Concretely: markers older than two minutes fade rather than persisting silently; an object whose heading was never transmitted is drawn as a disc rather than an arrow pointed north; the ship layer's subtitle names its actual coverage; and the satellite layer reports no data age at all, because a computed position does not have one and displaying "never" reads as a fault.

2.4 Schedule Feasibility

2.4.1 Approach

The project was delivered in phases, each with an explicit boundary. Work proceeded in sessions, and every session ended with a written handover so that no context depended on memory.

Phase	Delivered
1	Aircraft layer end to end: ingestion, API, globe, selection, detail panel
2	Satellite layer; migration from a 3D globe to MapLibre; deletion of the old renderer

- 3 Moon and solar system views
- 4 Accounts, tiers, advertisements, entitlements
- 5 Ship layer, regional then global
- 6 Performance and defect sweep

2.4.2 Evidence that the schedule held

The strongest evidence is the cost of the later phases. If the phase 1 architecture had been wrong, adding a class of object would have required rework across all three layers.

Measured:

- **The satellite layer** cost one provider module and one registry entry.
- **The ship layer** cost the same, four months later.
- **The polling logic was not modified for either**, because it reads a configuration value and does not know what it is fetching.

2.4.3 Where the schedule was lost, and to what

Reported honestly, because it is the more useful half.

Six sessions were spent on one defect that was misdiagnosed five times. The symptom was a rendering fault in the solar system view. Each of the five attempted fixes removed something genuinely wrong and left the symptom standing. The cause was structural: one camera was serving two pictures at wildly different scales, and every fix was a negotiation between them. The eventual resolution deleted 376 lines and added 130, and cost no feature at all.

Tooling consumed real time. The single largest recurring cost was stale development state — a hot-reloading server silently serving old code, producing measurements that described code no longer on disk. The lesson is recorded as working practice: *verify the instrument is attached to the thing before believing it about the thing.*

2.5 Related Systems and Research

2.5.1 Comparable systems

System	Covers	Model	Relationship to Orbital
Flightradar24	Aircraft	Commercial, freemium	The reference for what a mature aircraft tracker looks like. Far larger receiver network; closed data
MarineTraffic (Kpler)	Ships	Commercial	The equivalent for shipping. Acquired during this project
OpenSky Network	Aircraft	Research, non-commercial	Used as a <i>source</i> . Its non-commercial clause directly constrains Chapter 2.2
adsb.lol	Aircraft	Community,	Used as the primary aircraft source

AISHub	Ships	open Reciprocal	Evaluated and rejected: access requires contributing a receiver, which the team does not have
N2YO / Heavens-Above	Satellites	Free, ad-supported	Comparable satellite tracking; neither combines classes on one map

The distinguishing feature of Orbital is not any single layer, each of which is done better elsewhere by organisations with larger networks. It is that **three classes of object with three different acquisition models — observed and metered, observed and free, and computed — are presented on one map through one shared data shape**, and that the interface is explicit about the limits of each.

2.5.2 A market finding

A deliberate search was made for a free, global, key-less ship feed equivalent to adsb.lol's role for aircraft. **No such thing exists**, and the reason is structural rather than accidental: an AIS receiver is a real installation rather than a low-cost dongle, so volunteer networks operate on reciprocity — the aggregated feed in exchange for contributing one. The commercial half of the market has consolidated under two owners.

This is recorded so that the search is not repeated. The nearest equivalent, aisstream.io, is free and global but carries the unresolved licence question noted above.

2.6 Relevant Theories and Technologies

2.6.1 ADS-B — Automatic Dependent Surveillance–Broadcast

Aircraft broadcast identity, position, altitude and velocity roughly once a second. Coverage depends entirely on a receiver being in range, which is why oceanic and remote regions are sparse in every terrestrial network.

2.6.2 AIS — Automatic Identification System

Vessels broadcast position every few seconds and their *identity* — name, type, dimensions, destination — on a separate message every six minutes. Two consequences shape the ship layer:

“Not available” is a number, not a blank. A speed of 102.3 knots, a course of 360° and a heading of 511 all mean “the transmitter did not say”. In one live sample these covered 11, 88 and 142 vessels of 916. Rendered without interpretation they produce a moored ship travelling at 190 km/h.

Identity accumulates far more slowly than position. Measured on the live stream at 26.5 static messages per second against 114 position reports, vessel type coverage climbed from 3% at thirty seconds to **62% after fifty-four minutes**, and was still rising. This is a property of the protocol, not of the implementation.

2.6.3 SGP4 and orbital elements

Satellite positions are computed from published two-line element sets using the SGP4 propagator, the standard model for the format. Two properties matter:

- Elements describe an *orbit*, not a position, and remain usable for days — so the satellite layer keeps working through an upstream outage.
- **Accuracy degrades roughly a kilometre per day from epoch, silently.** Handed a 1975 element set, SGP4 returns a confidently formatted and entirely wrong answer. Of 1,670 sets in one live feed, **87 were over a year old and the oldest was from 1975.** Elements older than seven days are refused at ingestion.

2.6.4 Map projection and level of detail

The Web Mercator projection underlies both the globe and the street-level view. Two consequences were measured and are used directly:

- A pixel covers a different ground distance at every zoom and latitude. At zoom 7 a pixel is about 750 m; at zoom 14 about 6 m. This arithmetic determines how often the display needs redrawing at all — a ship crossing a pixel every two minutes does not need redrawing sixty times a second.
- Drawing every object at every zoom is neither useful nor affordable, which motivates the spatial thinning described in Chapter 3.

2.6.5 Signed distance field icons

Marker symbols are rendered as signed distance fields so that one silhouette can be tinted per object rather than requiring an image per colour. The trade-off is specific: the alpha channel encodes *distance*, not coverage, so thin detail does not merely shrink when scaled down — it dissolves, and the halo floods the cell. Marker artwork is therefore deliberately chunky.

2.6.6 HTTP conditional requests

The polled endpoints implement ETag-based conditional requests. When the underlying data has not changed, the server returns **304 Not Modified** before reading its store — no filtering, no thinning, no serialisation, no compression. The tag is *weak*, because two responses generated from one store version differ only in a reported age.

Chapter 3: Requirements Analysis

3.1 Stakeholder Analysis

Orbital's stakeholder map has an unusual shape, and recognising it early shaped the design: **most of the project's stakeholders are upstream, not downstream.** The parties with the greatest power over whether the system works are the data providers, none of whom have any relationship with the project or any obligation to it.

3.1.1 Upstream stakeholders — the data providers

Stakeholder	Interest	Influence	How the design responds
adsb.lol and its volunteer operators	That clients do not abuse a donated service	High — the primary aircraft source	One backend makes a fixed number of requests regardless of viewers; a request gate enforces the measured rate limit; every request identifies the project
OpenSky Network	Non-commercial use only; quota respected	High	Used as a supplement, not the primary; spending budgeted at 77% of allowance; startup refuses a configuration that would overspend
Fintraffic (Digitraffic)	Attribution under CC BY 4.0	Medium	Attributed; polled at the cadence its own cache headers request
aisstream.io	Undeclared	Medium	Used as a free public service and credited; behind a switch that disables it if the project is ever monetised
CelesTrak / SatNOGS	Not to be polled excessively	Medium	Elements refreshed every six hours, cached to disk, with a fallback source
Volunteer receiver operators	Recognition that coverage is <i>theirs</i>	Low individually	The map draws a receiver-coverage layer showing where no network reaches — their absence made visible

The most important consequence. Because the upstream stakeholders hold the power and grant no guarantees, the system is designed so that *any* of them failing degrades the display rather than breaking it. That is a requirement derived from stakeholder analysis, not from a technical preference.

3.1.2 Downstream stakeholders — the users

Stakeholder	Needs	Influence
Anonymous	To look without registering; not to be misled	High — the default

viewer	about accuracy	user
Registered free user	A saved identity; recent history	Medium
Premium user	A wider historical window	Medium
Administrator	Account management	Low — one role, no interface

An honest finding about the administrator role. Three tiers exist in the data — `free`, `premium` and `admin` — but the administrator has **no interface of its own**. An account can only be promoted to `admin` from a terminal on the server, and the self-service endpoint refuses that tier whatever it is asked. It is stated here plainly because it is a finding about the design rather than a gap to conceal: a tier reachable from the public interface is a tier anybody can have.

3.1.3 Project stakeholders — the team

Four members covering seven roles (CSC480):

Member	ID	Roles
Phone Sett Paing Kyaw	6708369	Project Manager (primary), Developer (primary)
Bhone Pyae Hein	6708381	System Analysis (primary), Co-Developer
Han Phyoo Htet	6708463	Quality Assurance (primary), Business Analysis (primary), Co-Tester
Nyan Lin Htet	6708397	Technical Engineer (primary), Tester (primary)

3.1.4 Power–interest summary

- **High power, high interest:** adsb.lol, OpenSky — manage closely; their terms are hard constraints on both architecture and revenue model.
- **High power, low interest:** CelesTrak, NASA GIBS, OpenFreeMap — keep satisfied; assume no support and design fallbacks.
- **Low power, high interest:** end users, the project team.
- **Low power, low interest:** the wider open-data community, which benefits from the recorded decisions.

3.2 Functional Requirements

Requirements are grouped by capability. **Priority** is M (must), S (should) or C (could). **Status** reflects the delivered system.

3.2.1 Data acquisition

ID	Requirement	Pri	Status
FR-	Retrieve live aircraft positions from at least one external source	M	Done

01			
FR-02	Merge two aircraft sources into one view, preferring the fresher record	S	Done
FR-03	Compute satellite positions from published orbital elements	M	Done
FR-04	Refuse orbital elements older than seven days	M	Done
FR-05	Retrieve live ship positions from a regional and a global source	S	Done
FR-06	Merge ship sources so a field is never overwritten by a blank	M	Done
FR-07	Poll each source on a configurable schedule within its rate limit	M	Done
FR-08	Retain the last successful snapshot when a source fails	M	Done
FR-09	Evict objects not re-observed within a per-layer time-to-live	M	Done
FR-10	Run entirely offline from committed fixture data	M	Done

3.2.2 API

ID	Requirement	Pri	Status
FR-11	List objects of a layer, filtered by bounding box	M	Done
FR-12	Return one object in full, with its observed track	M	Done
FR-13	Search a layer by name or identifier	M	Done
FR-14	Search aircraft, airports, satellites and ships in one request	S	Done
FR-15	Reduce results to a configured maximum, spread across the visible area	M	Done
FR-16	Answer 304 Not Modified when the client holds the current representation	S	Done
FR-17	Report ingestion health and data freshness	M	Done
FR-18	Never call an external service while handling a request	M	Done
FR-19	Refuse malformed input with a precise error	M	Done
FR-	Clamp a requested result count to the configured maximum	M	Done

3.2.3 Presentation

ID	Requirement	Pri	Status
FR-21	Draw an interactive Earth, rotatable and zoomable from globe to street level	M	Done
FR-22	Draw each object at its position, oriented to its heading	M	Done
FR-23	Draw an object whose heading is unknown without implying one	M	Done
FR-24	Fade an object whose position is more than two minutes old	M	Done
FR-25	Interpolate positions between polls	M	Done
FR-26	Switch between the aircraft, satellite and ship layers	M	Done
FR-27	Change all surrounding text, the key and the search hint with the layer	M	Done
FR-28	Select an object and show its details	M	Done
FR-29	Draw the observed track of the selected object	M	Done
FR-30	Draw the selected aircraft as a three-dimensional airframe	C	Done
FR-31	Show receiver coverage — the regions no network reaches	S	Done
FR-32	Offer a solar system view with navigable planets	C	Done
FR-33	Colour each layer by the property that varies within it	M	Done

3.2.4 Accounts and entitlements

ID	Requirement	Pri	Status
FR-34	Register with an email address and password of at least 10 characters	M	Done
FR-35	Sign in and out; maintain a session across reloads	M	Done
FR-36	Store passwords using a memory-hard hash, never in plain text	M	Done
FR-37	View satellite positions at a past instant	S	Done

FR-38	Limit the historical window by tier — 24 hours free, 7 days premium	M	Done
FR-39	Refuse a request outside the tier's window with a stated reason	M	Done
FR-40	Allow a signed-in account to change its own tier between free and premium	S	Done
FR-41	Refuse the administrator tier from the public interface under all conditions	M	Done
FR-42	Show advertisements to accounts that are not premium	S	Done

Note on FR-38. The premium window is seven days because that is the point at which SGP4 drift makes the answer untrustworthy — the limit is physics, and the tier is sold up to it rather than beyond it. No tier can exceed that window.

3.3 Non-Functional Requirements

Each is stated with the measurement that verifies it.

3.3.1 Performance

ID	Requirement	Target	Measured
NFR-01	Marker rendering at the display cap	< 16.7 ms/frame	0.80 ms at 2,000 markers
NFR-02	Sustained frame time under load	60 fps	4.2 ms p50, 5.9 ms p99 at zoom 7 with 2,000 vessels
NFR-03	List endpoint response	< 100 ms	23 ms
NFR-04	Combined search response	< 100 ms	47 ms
NFR-05	Detail lookup	< 10 ms	0.01 ms
NFR-06	Response payload compressed	—	366 KB to 24.5 KB
NFR-07	No request may block the event loop indefinitely	—	Result count clamped; unbounded request removed

NFR-02 is the requirement that drove the largest single optimisation. The display initially rebuilt its entire drawing data on every frame, costing 7.4 ms at the median and 43 ms at the tail. Because a pixel at zoom 7 covers 750 m, and a ship crosses one every two minutes, almost all of that work was animating motion below the resolution of the screen. Deriving the redraw interval from the view reduced p99 frame time from **55.1 ms to 5.9 ms**.

3.3.2 Reliability

ID	Requirement	Verified by
NFR-08	An upstream failure must not produce an error response	Injected outages; a real day-long CelesTrak outage
NFR-09	Data older than its threshold must be flagged, not hidden	<code>stale</code> flag on every list response
NFR-10	One source failing must not disable a multi-source layer	Partial-failure tests on both unions
NFR-11	A dropped stream must reconnect without losing accumulated state	Reconnection tests; observed live
NFR-12	The system must start without credentials	Fixture provider is the default

3.3.3 Accuracy and honesty

This category is treated as a first-class requirement rather than a matter of presentation, because the failures it prevents are ones a viewer cannot detect.

ID	Requirement
NFR-13	A protocol's "not available" encoding must never be displayed as a value
NFR-14	A position must never be shown at a default coordinate; an object without a position is not drawn
NFR-15	An unknown heading must not be drawn as north
NFR-16	A value must be shown in the unit its readers use — knots at sea, metres per second in the contract
NFR-17	Text surrounding the map must describe the layer actually shown, including its coverage limits
NFR-18	A computed position must not report an observation age
NFR-19	A speed physically impossible for the object must be rejected, not displayed

3.3.4 Security

ID	Requirement
NFR-20	Passwords hashed with <code>bcrypt</code> ; never logged or returned
NFR-21	Session cookies <code>HttpOnly</code> and <code>SameSite</code> ; <code>Secure</code> available for deployment
NFR-22	Credentials read from environment or a git-ignored file, never committed
NFR-23	A configuration object must never be printed whole, since it may contain a secret
NFR-24	Administrator privilege must not be obtainable through any public endpoint

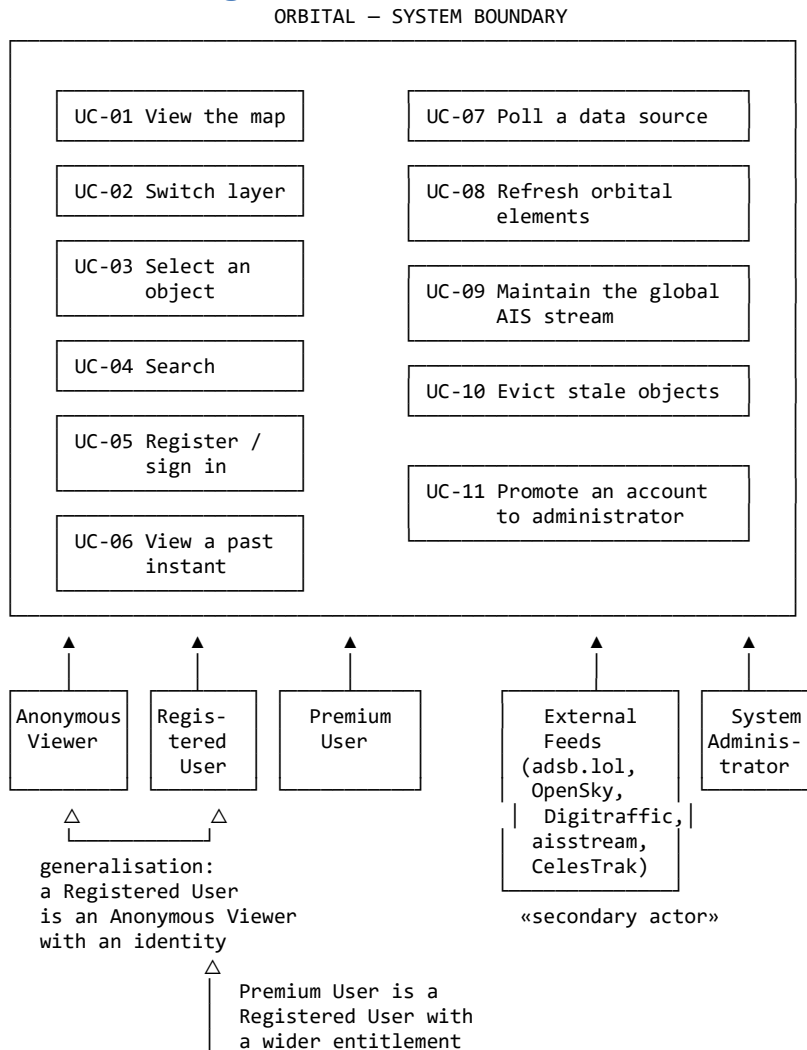
3.3.5 Maintainability

ID	Requirement	Evidence
NFR-25	One shared data shape across all layers and both languages	<code>models.py</code> and <code>types.ts</code> , kept in step
NFR-26	A new data source must not require frontend changes	Demonstrated twice
NFR-27	A new object class requires a written decision	Enforced by a test on each side
NFR-28	Every significant decision recorded with alternatives and measurements	166 entries
NFR-29	Both suites runnable offline	Fixture provider; no test makes an external call

3.3.6 Usability and accessibility

ID	Requirement
NFR-30	Usable without an account
NFR-31	Reduced-motion preference respected in every animation
NFR-32	Interactive controls reachable and labelled for assistive technology
NFR-33	Layout usable at a 375 px viewport width

3.4 Use Case Diagram



Relationships. UC-03 Select an object **«includes»** UC-01 View the map. UC-06 View a past instant **«extends»** UC-01, at the point where an instant is chosen, under the condition that the requested instant lies within the actor's entitlement window. UC-07, UC-08, UC-09 and UC-10 are initiated by a scheduler rather than a person, and are shown because they are where every external dependency and every failure mode lives.

3.5 Use Case Descriptions

UC-01 — View the map

Actor	Anonymous Viewer
Goal	See where things are now
Preconditions	None. No account is required
Trigger	The viewer opens the application

Main flow 1. The system draws the Earth on satellite imagery. 2. The client requests the active layer's objects for the visible area. 3. The API returns the cached snapshot, filtered and thinned, with its age. 4. The client draws each object oriented to its heading, coloured by the property that varies on that layer. 5. Between polls the client interpolates positions. 6. The status bar reports the count and the age of the data.

Alternate flows - A1 — data is stale. The response is flagged `stale`; the status bar shows the age and markers past two minutes are drawn faded. - **A2 — the backend is unreachable.** The last received state remains on screen; the status bar reports the failure. The map is not cleared. - **A3 — more objects than the cap.** The API returns a spatially even sample and reports both figures: "2,000 ships, showing a sample of 23,297 in view".

Postcondition — the viewer sees current positions, or is told why not.

UC-02 — Switch layer

Actor Anonymous Viewer

Goal Look at a different class of object

Main flow 1. The viewer selects Aircraft, Satellites or Ships. 2. The system clears the current selection and objects. 3. **All surrounding text changes with the layer** — subtitle, search placeholder, the count noun, the colour key and its heading. 4. Layer-specific furniture is hidden — airport markers and receiver coverage belong to the aircraft layer and are meaningless over open sea. 5. The client polls the new layer's endpoint.

Postcondition — everything on screen describes the layer now shown.

UC-03 — Select an object

Actor Anonymous Viewer

Includes UC-01

Main flow 1. The viewer clicks a marker. 2. The system identifies the object under the pointer. 3. The client requests that object's full record. 4. A panel opens with fields appropriate to the layer — an aircraft shows airline, type and route; a ship shows vessel type, navigational status, speed in knots, destination and dimensions; a satellite shows its orbit. 5. The object's observed track is drawn.

Alternate flows - A1 — empty map clicked. The selection is cleared and the panel closes. - **A2 — object no longer tracked.** A 404 is returned; the panel reports that it is no longer being tracked rather than showing an empty record.

UC-04 — Search

Actor Anonymous Viewer

Goal Find a specific object or place by name

Main flow 1. The viewer types into the search box. 2. After a debounce, the client requests results. 3. The system returns matches in separate groups — aircraft, airports, satellites, ships — ranked within each but never ranked *across* groups, because they are not comparable. 4. **Only the groups the active layer can act on are shown**, since offering a result that would throw the viewer out of their layer answers a question nobody asked. 5. Choosing a result moves the camera to it and selects it.

Alternate flow - A1 — no match. A message states why, naming the limits of that layer.

UC-05 — Register and sign in

Actor Anonymous Viewer → Registered User

Main flow 1. The viewer opens the sign-in page and supplies an email and password. 2. The system rejects a password shorter than ten characters. 3. The password is hashed with bcrypt; the plain text is never stored or logged. 4. A session cookie is issued — HttpOnly and SameSite. 5. The viewer is returned to the map, now signed in.

Alternate flows - A1 — email already registered. Reported without revealing whether the password was correct. - **A2 — wrong credentials.** One message for both wrong email and wrong password, so the response does not disclose which accounts exist.

UC-06 — View a past instant

Actor Registered User; Premium User

Extends UC-01

Precondition Signed in; satellite layer active

Main flow 1. The user moves the time control to an instant in the past. 2. The client requests satellite positions for that instant. 3. The system checks the instant against the account's entitlement window — **24 hours for free, 7 days for premium**, with a small grace margin. 4. Positions are propagated to the requested instant and returned.

Alternate flows - A1 — outside the window. The request is refused with a stated reason and the window that does apply. The free user is shown what premium offers. - **A2 — beyond seven days.** Refused for **every** tier, including administrators, because propagation accuracy degrades roughly a kilometre per day and beyond a week the answer would be confidently wrong.

UC-07 — Poll a data source (*system*)

Actor Scheduler (initiating), External Feed (secondary)

Goal Keep the cache current without exceeding any source's limits

Main flow 1. The scheduler waits the configured interval for the job. 2. It checks the remaining quota and the rate gate; if either forbids, it skips and reports the skip. 3. The provider requests data and converts it to the shared shape, discarding any object without a usable position and converting every “not available” encoding to an explicit absence. 4. The store merges the result and records the time.

Alternate flows - **A1 — source unreachable or 5xx**. The failure is recorded, the previous snapshot is retained, and the next attempt is delayed by an increasing backoff. - **A2 — rate limited**. The upstream’s own retry hint is used. - **A3 — both sources of a layer fail**. Only then is the poll an outage.

UC-08 — Refresh orbital elements (*system*)

Main flow 1. Every six hours, elements are fetched from CelesTrak. 2. Element sets older than seven days from epoch are refused. 3. Accepted elements are cached to disk so a restart during an outage still has something to propagate.

Alternate flow - **A1 — CelesTrak unavailable**. SatNOGS is used instead. If both fail, the cached elements continue to serve; they remain accurate for days. *This path is not hypothetical — CelesTrak returned 503 for an entire working day.*

UC-09 — Maintain the global AIS stream (*system*)

Main flow 1. A WebSocket connection is opened and subscribed to the whole world. 2. Messages are decoded continuously; positions and identities are accumulated in memory. 3. Positions older than fifteen minutes are removed; identities are kept for six hours, because a vessel’s name does not expire when its position does.

Alternate flow - **A1 — connection dropped**. Reconnection with backoff. Nothing is lost: the accumulated world is held locally, and the service does not replay.

UC-11 — Promote an account to administrator (*system*)

Actor System Administrator, at a terminal on the server

Main flow 1. The administrator runs the account command-line tool on the host. 2. The tool identifies the account and sets its tier.

Constraint — there is **no interface** for this and no endpoint that can perform it. The self-service tier endpoint refuses `admin` under all configurations, giving the same response as for a nonsensical tier.

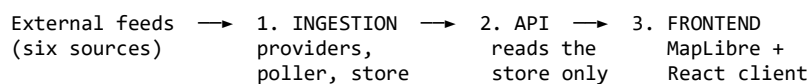
3.6 Software Requirements Specification (SRS)

3.6.1 Purpose and scope

This specification covers Orbital, a web application presenting live aircraft, ship and satellite positions on an interactive Earth. It defines the external interfaces, the shared data contract, and the constraints under which the system operates. It does not cover the deployment environment, which is out of scope for this phase.

3.6.2 Overall description

Product perspective. Orbital is a three-layer system with one rule between the layers: data flows in one direction, and each layer knows only the layer directly beneath it.



Three properties follow, and they are the design:

1. **The browser never talks to a data source.** Rate limiting and caching are enforced in exactly one place.
2. **Ingestion does not know a browser exists, and the frontend does not know where the data came from.** Either can be replaced without touching the other, and both have been.
3. **Upstream failure is contained at layer 1.** The API serves the last good snapshot with an explicit staleness flag.

User classes. Anonymous Viewer (default, full read access); Registered User (identity, 24-hour history); Premium User (7-day history, no advertisements); Administrator (terminal only).

Constraints.

Constraint	Source
OpenSky may not be used commercially	Provider terms
OpenSky spending must stay within a daily credit allowance	Provider terms
adsb.lol tolerates 4 requests then ~1 per 12 s	Measured
Digitraffic states a 60-second cache; polling faster returns the same body	Provider headers
Satellite accuracy is untrustworthy beyond 7 days from epoch	SGP4
The backend is a single-threaded event loop	Runtime
AIS identity arrives on a 6-minute cycle	Protocol

Assumptions and dependencies. The system assumes continuous availability of none of its sources; every dependency has a defined degraded mode.

3.6.3 External interface requirements

User interfaces. A single-page application: a full-viewport map; a header carrying the wordmark, search and layer toggle; a collapsible colour key; a status bar reporting count, sample size and data age; a detail panel; and full-screen pages for sign-in, premium and the solar system, each addressable by a URL fragment so the browser’s Back button dismisses it.

Software interfaces — the API.

Method	Path	Purpose
GET	/api/aircraft	Aircraft in a bounding box
GET	/api/aircraft/{id}	One aircraft with its track and route
GET	/api/aircraft/search	Aircraft by callsign or address
GET	/api/satellites	Satellites, optionally at a past instant
GET	/api/satellites/{id}	One satellite
GET	/api/ships	Ships in a bounding box
GET	/api/ships/{id}	One ship
GET	/api/ships/search	Ships by name or MMSI
GET	/api/search	Combined search across four kinds
GET	/api/moon/satellites	Spacecraft in lunar orbit
GET	/api/health	Ingestion status and freshness
POST	/api/auth/register	Create an account
POST	/api/auth/login	Begin a session
POST	/api/auth/logout	End a session
GET	/api/auth/me	The current account
POST	/api/auth/subscription	Change own tier (never to admin)

The shared data shape. Every moving object, whatever its source, is represented identically:

Field	Type	Meaning
id	string	Stable identifier within a provider
lat, lon	number	Degrees, WGS84
altitude	number null	Metres above mean sea level. Zero means zero — a ship is at sea level, which is a fact and not a gap. <code>null</code> means the source did not say
velocity	number null	Metres per second
heading	number null	Degrees clockwise from true north. <code>null</code> is never rendered as north
label	string	Short human-readable name

<code>model</code>	string null	What the source says the object <i>is</i> , in its own vocabulary
<code>lastSeen</code>	string	When the position was current — never when we polled
<code>type</code>	enum	aircraft satellite ship

Anything meaningful to only one kind of object lives in a generic `meta` map, so that a provider can add a field without any frontend change. This single decision is what made the second and third layers cost one module each.

3.6.4 Functional requirements

As specified in §3.2 (FR-01 to FR-42).

3.6.5 Non-functional requirements

As specified in §3.3 (NFR-01 to NFR-33).

3.6.6 Verification

Requirement class	How verified
Functional	1,842 automated tests; live runs against every real source
Performance	Instrumented measurement in the browser and a backend benchmark
Reliability	Injected outages, plus real ones that occurred unprompted
Honesty (NFR-13–19)	Unit tests on each conversion, plus inspection of the running application
Security	Tests on hashing, session handling and tier refusal

A note on verification method. A recurring finding across this project is that a passing test suite is not sufficient evidence. Eighteen defects are recorded as having been invisible to the tests at the time they existed, each found by running the application and looking at it, and the most recent phase added several more — including a receiver-coverage layer drawn over Mars, a three-dimensional aeroplane rendered on the sea, and a ship layer that drew 37 vessels where 9,159 were present. In each case the code was correct and the wiring was absent or mismatched. Unit tests verify code; only running the system verifies wiring.